

Redes de Comunicaciones y Cómputo Distribuido



2do. Cuatrimestre de 2026

Licenciatura en Ciencias de la Computación
Departamento de Computación
Facultad de Cs. Exactas y Naturales
Universidad de Buenos Aires



Vinton Cerf
en Exactas-UBA
22-08-2007

Nivel de Transporte

TCP (Transmission Control Protocol)

Bibliografía Básica

- ▶ Computer Networks, A Systems Approach (Fifth Edition) (The Morgan Kaufmann Series in Networking) [Larry L. Peterson](#), [Bruce S. Davie](#) (2011)
 - ▶ Capitulo 5 Protocolos End to End –TCP págs. 396-431
 - ▶ Capitulo 6 Control de Congestión (CC) y Alocación de Recursos – págs. 479-499 y TCP CC págs. 499-530

Conveniente complementar con la lectura de:

- ▶ Computer Networks (5th. Edition) [Andrew S. Tanenbaum](#), [David J. Wetherall](#) (2010)
 - ▶ TCP: págs. 552-571, Algoritmos de Control de Congestión pags 392-404
 - ▶ Nivel de Transporte – CC págs. 530-541
 - ▶ TCP CC págs. 571-581

Referencias

- ▶ Cerf, V., and R. Kahn, "A Protocol for Packet Network Intercommunication", IEEE Transactions on Communications, Vol. COM-22, No. 5, pp 637-648, May 1974.
- ▶ [RFC 793] J. Postel "Transmission Control Protocol" September 1981. (varios errores e inconsistencias)
- ▶ [RFC 1122] October 1989 (se salvan bugs de la RFC 793)
- ▶ [RFC 6298] Paxson, Allman, Chu and Sargent, "Computing TCP's Retransmission Timer", June 2011
- ▶ [RFC 1323] Jacobson, V., Braden, R., and D. Borman, "TCP Extensions for High Performance", May 1992.
- ▶ [RFC 2018] Mathis, M., Mahdavi, J., Floyd, S., and A. Romanow, "TCP Selective Acknowledgment Options", October 1996.
- ▶ [RFC 2988] Paxson, V. and M. Allman, "Computing TCP's Retransmission Timer", November 2000.
- ▶ Una recopilación al 2006: "A Roadmap for Transmission Control Protocol (TCP) Specification Documents" RFC 4614

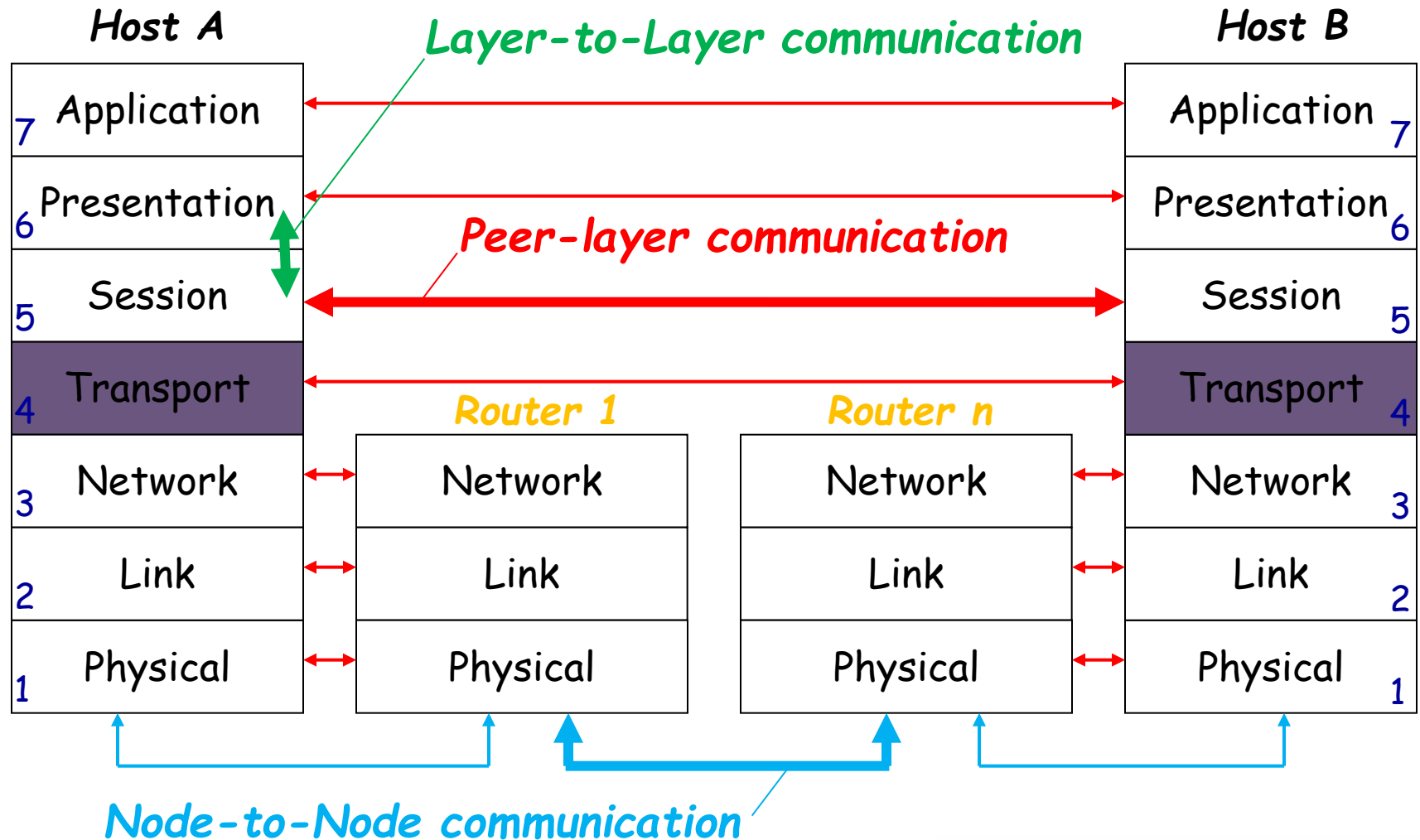
Agenda

- ▶ Nivel de Transporte
- ▶ El Protocolo TCP (Transmission Control Protocol)
 - TCP Connection setup
 - Segmentos TCP
 - Números de Secuencia TCP
 - TCP Sliding Window
 - Control de Flujo
- ▶ El Protocolo UDP (User Datagram Protocol)
- ▶ El Protocolo TCP - Dinámicas Especiales
 - Timeouts y Retransmisiones
 - Máquina de Estados Finitos
 - Ineficiencia por envío de segmentos pequeños y que el receptor los pida
 - Uso del ancho de banda
 - Sockets

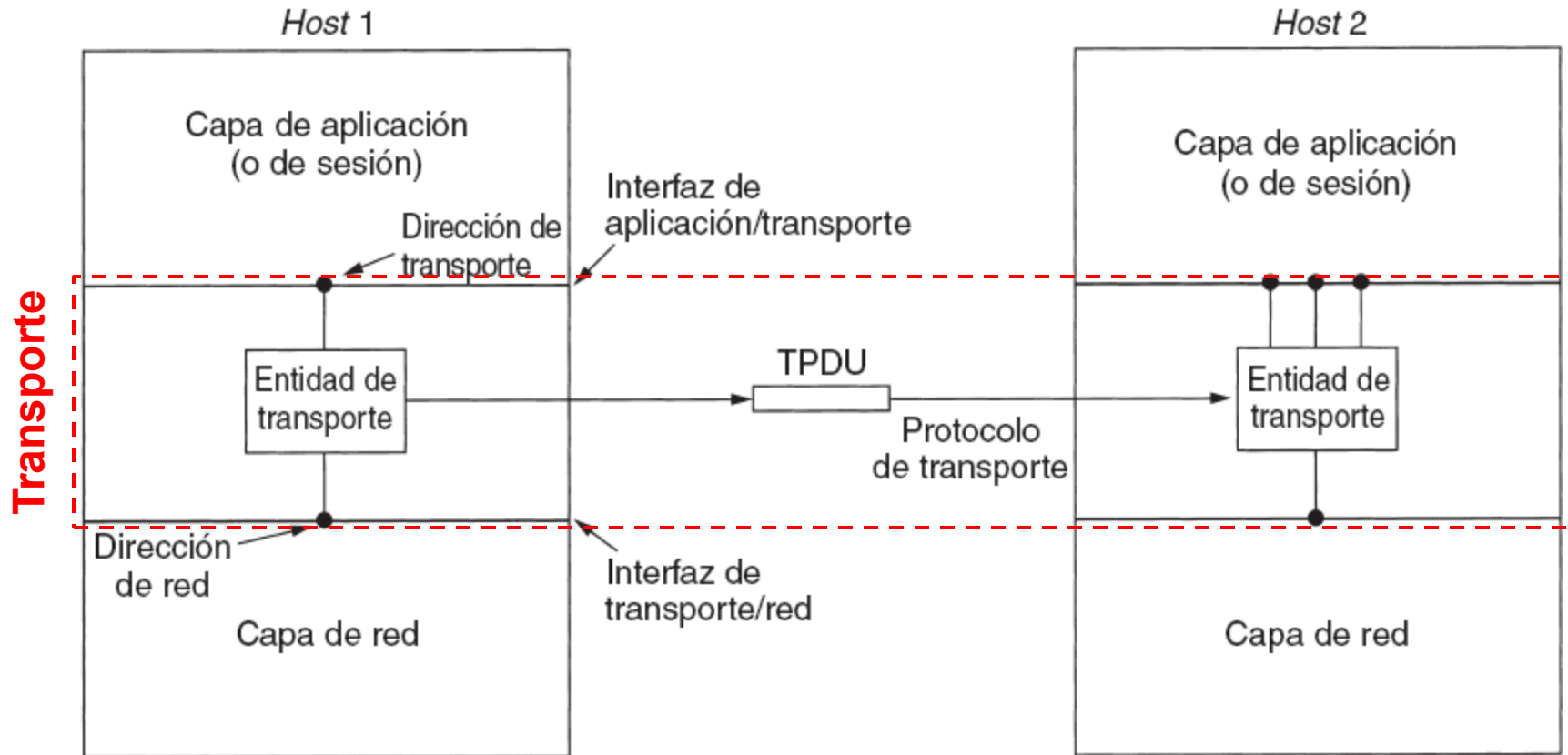
Repaso: los modelos de capas

Protocolos End to End (E2E)

Modelo OSI



Modelo OSI



TPDU = **T**ransport **P**rotocol **D**ata **U**nit

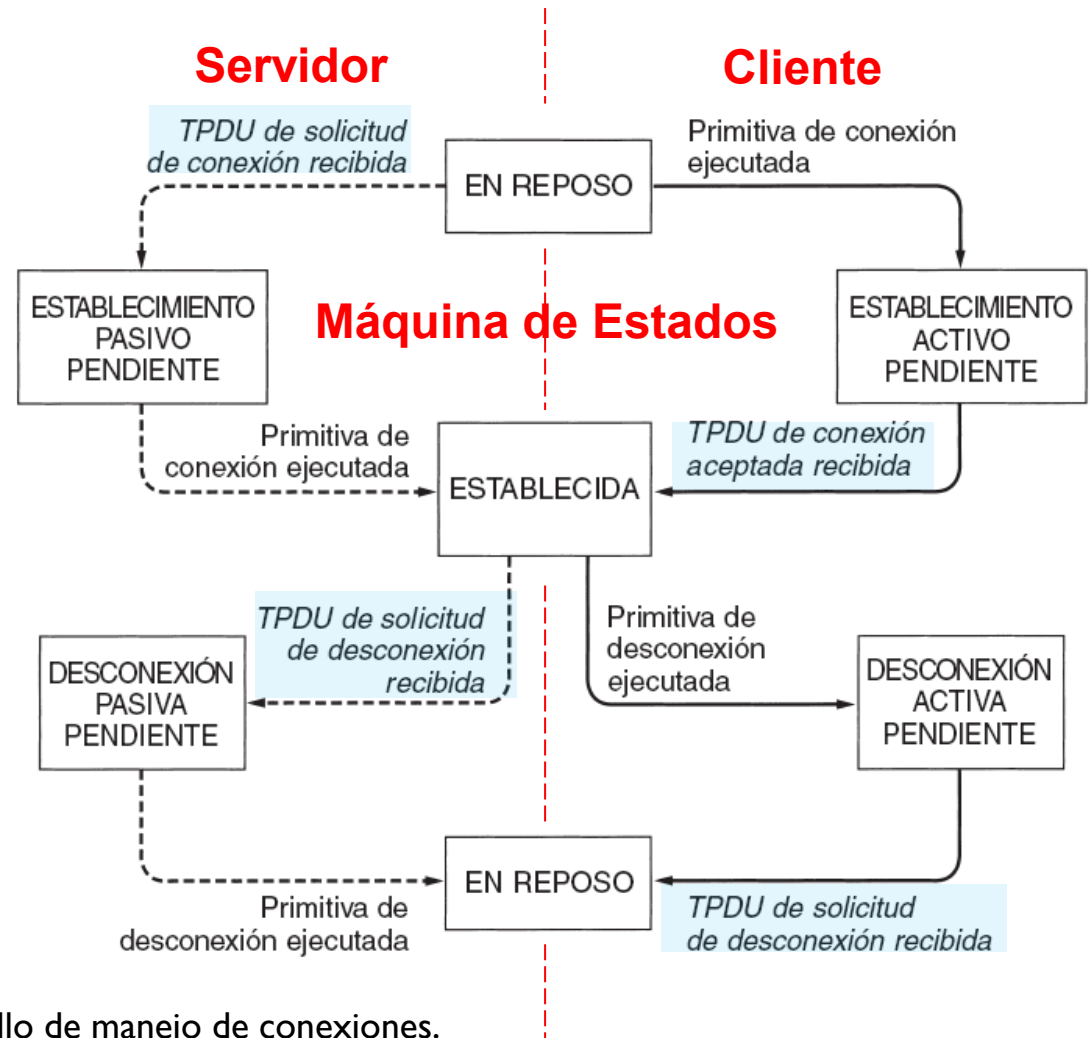
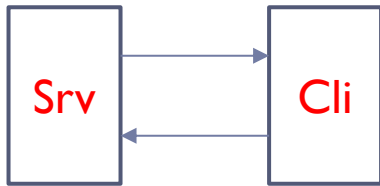
Enlace de Datos versus Transporte

Un servicio de **Transporte**:

- ▶ Potencialmente **conecta muchas máquinas diferentes**
 - ▶ requiere de **establecimiento y finalización de conexión** explícitos
- ▶ Potencialmente lidia con **diferentes RTT**
 - ▶ requiere mecanismos adaptivos para **timeouts**
- ▶ Potencialmente enfrenta **largos retardos en la red**
 - ▶ requiere estar preparado para el arribo de **paquetes muy antiguos**
- ▶ Potencialmente existe **diferente capacidad de recepción en destino**
 - ▶ requiere contemplar nodos de diferentes capacidades
- ▶ Potencialmente maneja **diferente capacidad de red**
 - ▶ requiere estar preparado para lidiar con **congestión de red**

Modelo OSI

TPDU =
Transport Protocol Data Unit



Máquina de Estados de un esquema sencillo de manejo de conexiones.

- Las **transiciones en cursiva** son causadas por **llegadas de paquetes**.
- Las líneas **continuas** muestran la secuencia de **transiciones de estado** del **Cliente**.
- Las líneas **punteadas** muestran la secuencia de **transiciones de estado** del **Servidor**.

Protocolos “End-to-End” en subredes de datagramas

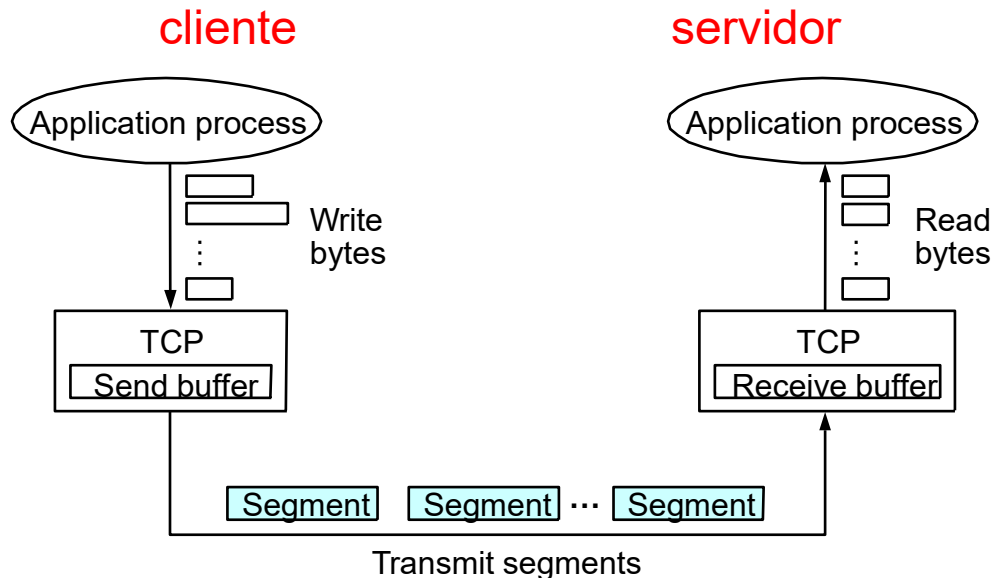
- ▶ Se apoyan en la **Capa de Red**, la cual es de “**mejor esfuerzo**” (**best-effort**)
 - ▶ descarta mensajes
 - ▶ desordena mensajes
 - ▶ puede entregar múltiples copias de un mensaje dado
 - ▶ limita los mensajes a algún tamaño finito
 - ▶ entrega mensajes después de un tiempo arbitrariamente largo
- ▶ Servicios comunes **End-to-End** ofrecidos/deseados
 - ▶ **garantía de entrega de mensajes (confiabilidad)**
 - ▶ entrega de mensajes en el mismo orden que son enviados
 - ▶ entrega de a lo sumo una copia de cada mensaje
 - ▶ soporte para mensajes arbitrariamente largos
 - ▶ soporte de sincronización
 - ▶ permitir al receptor controlar el flujo de datos del transmisor
 - ▶ soportar múltiples procesos de nivel aplicación en un receptor

TCP

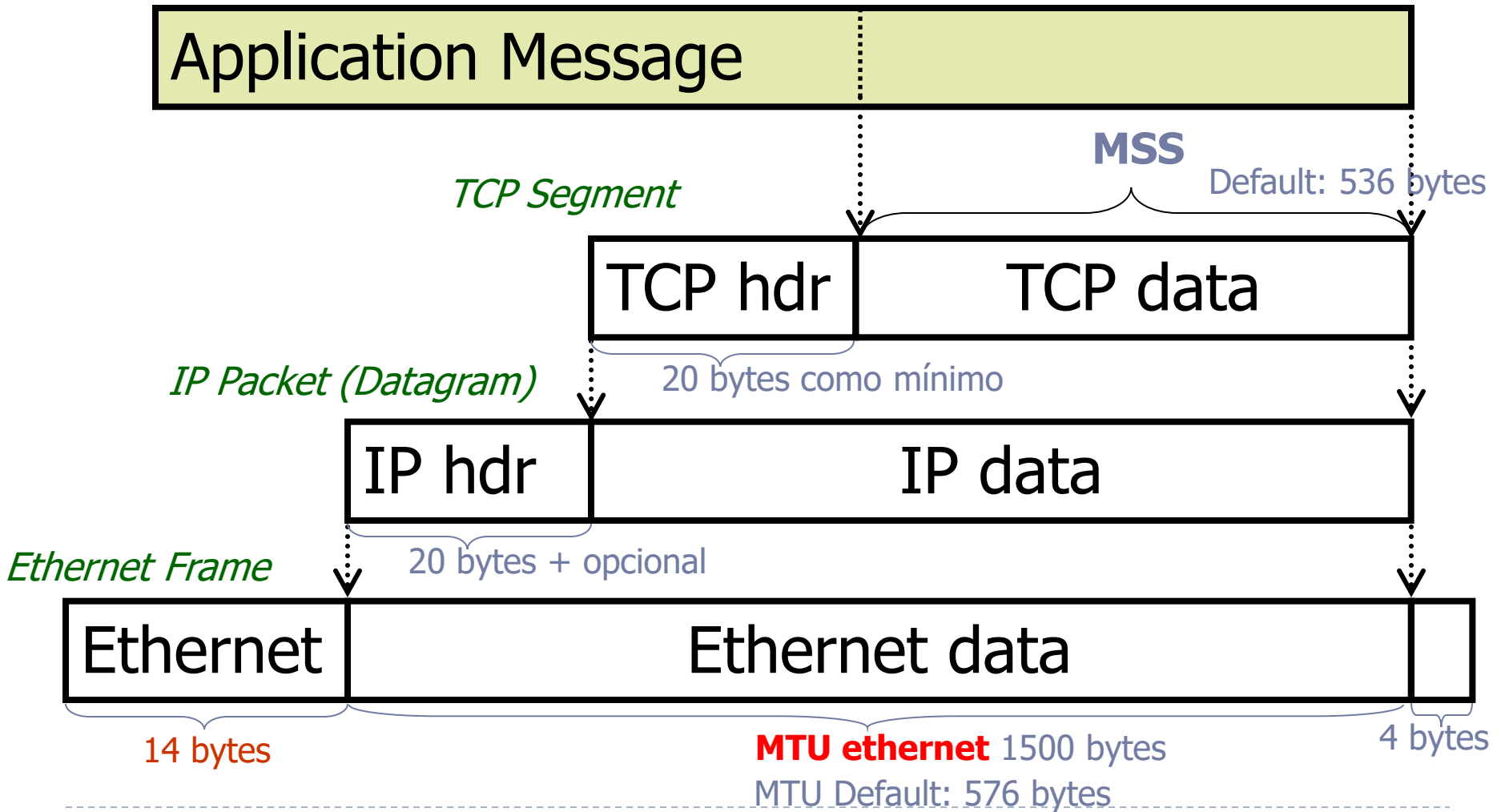
Características
Formato del Header

TCP - Generalidades

- ▶ Orientado a conexión
- ▶ Flujo de bytes
 - ▶ App escribe bytes
 - ▶ TCP envía *segmentos*
 - ▶ App lee bytes
- ▶ Full Duplex (dos flujos de bytes)
 - ▶ **Control de flujo**: evita que el Tx **inunde al Rx**
 - ▶ **Control de congestión**: evita que el Tx **sobrecargue a la red**



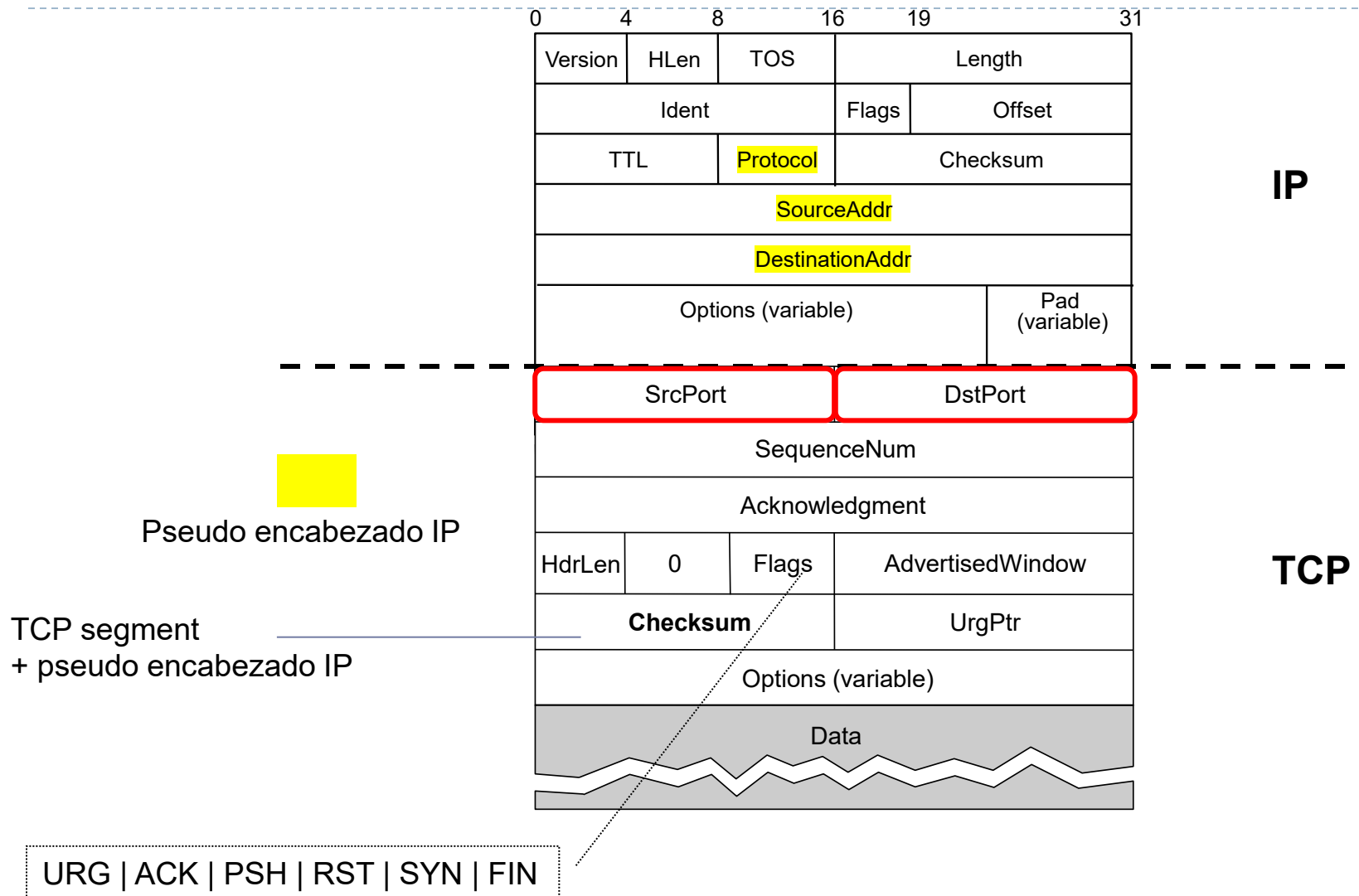
MSS: "Maximum Segment Size"



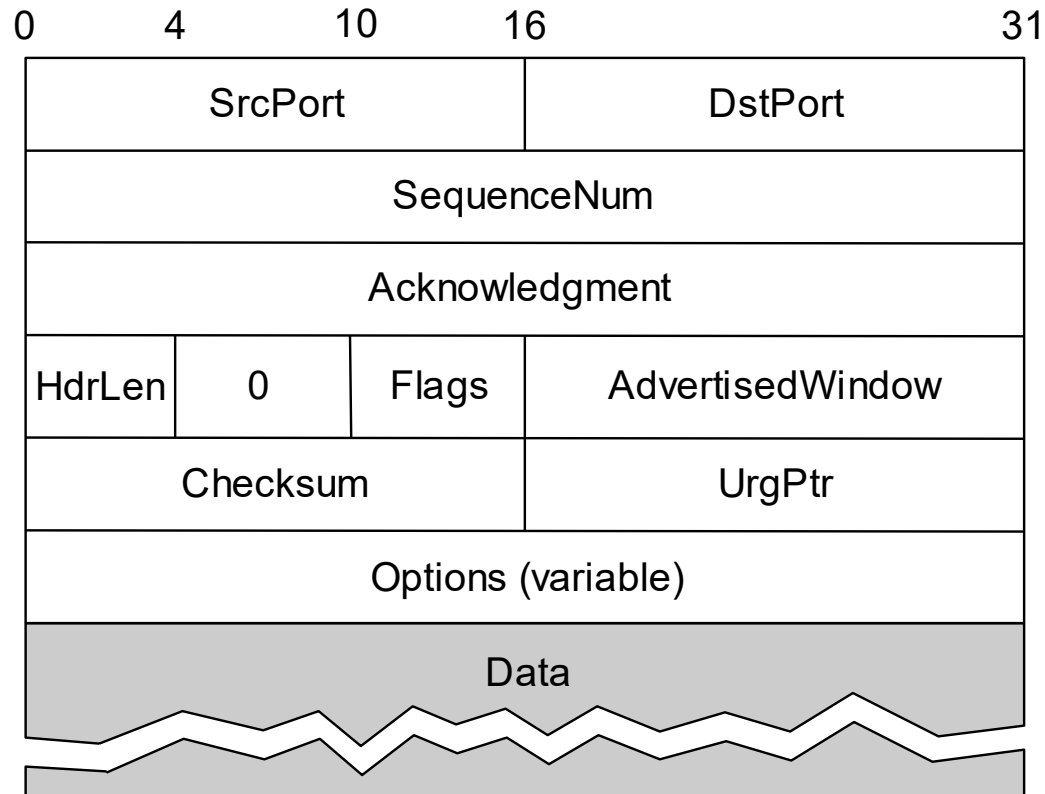
TCP: Características

- ❑ TCP es **orientado a conexión**
 - ❑ Manejo de la conexión:
 - ❑ 3-way handshake usado para **establecimiento (setup)**
 - ❑ 2-2 o 4-way handshake para la **liberación (release)**
 - ❑ TCP provee un servicio de flujo de bytes (***stream-of-bytes***)
- ❑ TCP es **confiable** (establece una “conexión lógica entre sockets”)
 - ❑ Acknowledgements (ACKs)
 - ❑ Checksums
 - ❑ Números de secuencia (para detectar datos perdidos o desordenados)
 - ❑ Datos perdidos o corruptos se ReTX después de un Timeout
 - ❑ Datos desordenados se podrían reordenar
 - ❑ Implementa **Control de Flujo** evita inundar al receptor
 - ❑ Implementa **Control de Congestión**
(***lo veremos en detalle una clase especial aparte***)

Contexto: Formato de Segmento

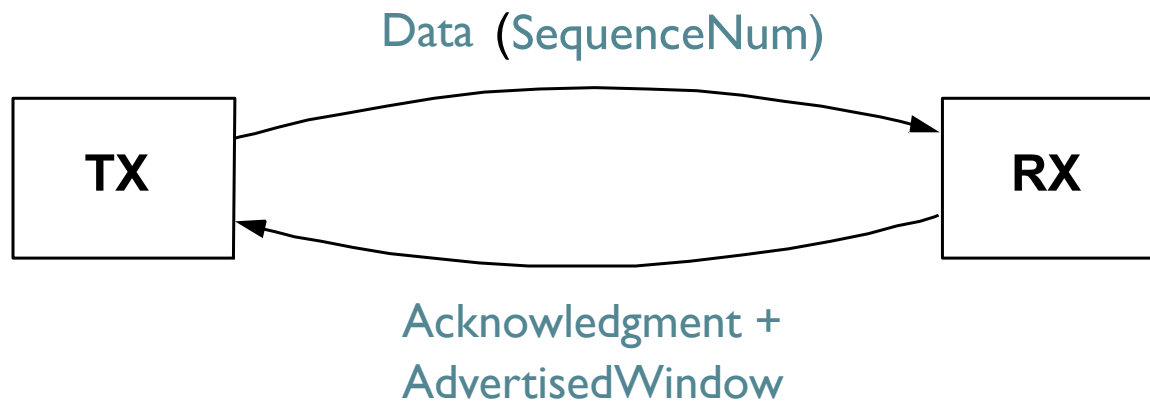


Formato de Segmento TCP



Conexión TCP

- ▶ Cada conexión es identificada por la 4-tupla:
 - ▶ **SrcPort, SrcIPAddr, DstPort, DstIPAddr**
- ▶ Ventana Deslizante + Control de Flujo
 - ▶ **Acknowledgement, SequenceNum, AdvertisedWindow**



- ▶ Flags
 - ▶ **SYN, FIN, RESET, PUSH, URG, ACK**
- ▶ Checksum
 - ▶ pseudo header (IP) + TCP header + TCP data

La cabecera de TCP

- ▶ **Puerto fuente y Puerto destino (16 bits):** Identifican los puntos finales locales de la conexión.
- ▶ **Número de secuencia (32 bits):** identifica de forma unívoca los datos de aplicación que contiene el segmento TCP. Identifica el primer *byte de datos*.
- ▶ **Número de reconocimiento (32 bits):** Indica el siguiente *byte que espera el receptor*.
 - ▶ Implica una confirmación (ACK) de *todos los bytes recibidos hasta ese momento.*
- ▶ **Longitud de cabecera (4 bits):** Indica cuántas palabras de 32 bits están contenidas en la cabecera de TCP.
 - ▶ Es necesario a causa de la longitud variable del campo Opciones.

La cabecera de TCP, Flags:

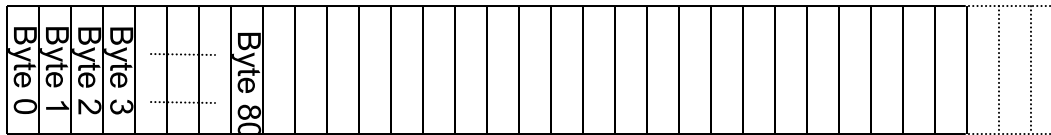
- ▶ **URG:** Es igual a 1 si el campo *Puntero a Urgente* está en uso.
- ▶ **ACK:** Es igual a 1 para indicar que el número de reconocimiento es válido.
 - ▶ Si vale 0, el paquete no contiene un reconocimiento, y entonces el campo *número de reconocimiento* es ignorado.
- ▶ **PSH:** “PUSHed data”. Indica al receptor que debe entregar los datos a la aplicación inmediatamente después del arribo del paquete.
 - ▶ No debe esperar hasta que un buffer total haya sido recibido.

La cabecera de TCP

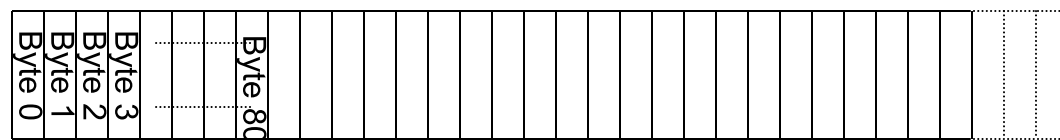
- ▶ **RST:** Se utiliza para resetear una conexión que se ha vuelto confusa debido a la caída de un host o alguna otra razón.
- ▶ **SYN:** Es utilizado para **establecer** conexiones.
 - ▶ En esencia es usado para indicar **Requerimiento de Conexión y Conexión Aceptada**.
- ▶ **FIN:** Es utilizado para **liberar** conexiones.
 - ▶ Especifica que el emisor no tiene más datos para transmitir.
- ▶ **Tamaño de la ventana:** Indica cuántos bytes pueden ser enviados comenzando desde el último byte reconocido.
 - ▶ Para propósitos de control de **flujo**.

TCP soporta un “*stream de bytes*”

Host A

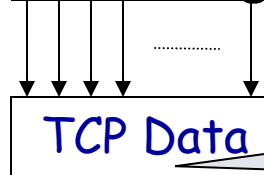
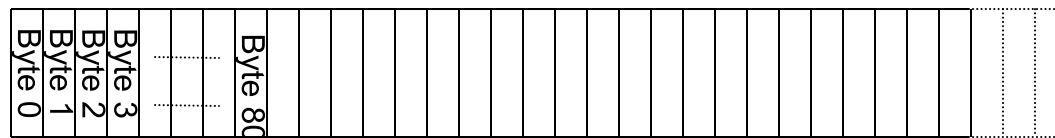


Host B



...se emula usando segmentos

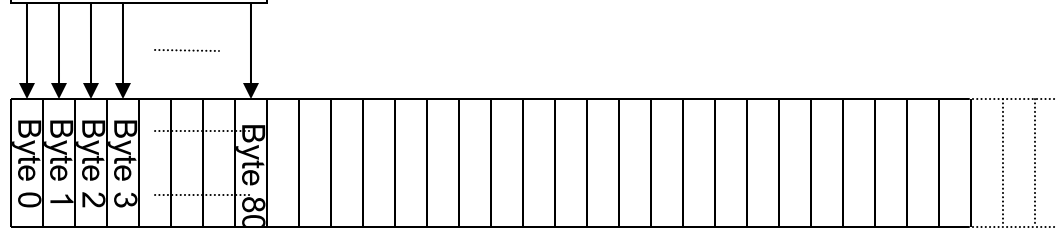
Host A



Un segmento se envía cuando:

1. Segmento full (MSS bytes)
2. No está "full", pero sucede TimeOut
3. "Pushed" por la aplicación.

Host B

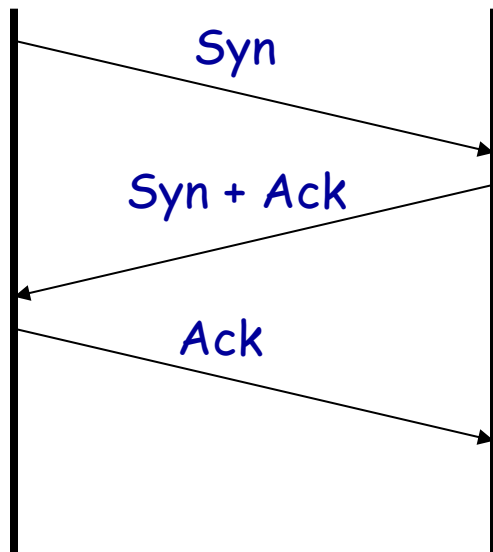


Establecimiento y Liberación de la Conexión TCP

Establecimiento

(Activo)
Cliente

(Pasivo)
Servidor

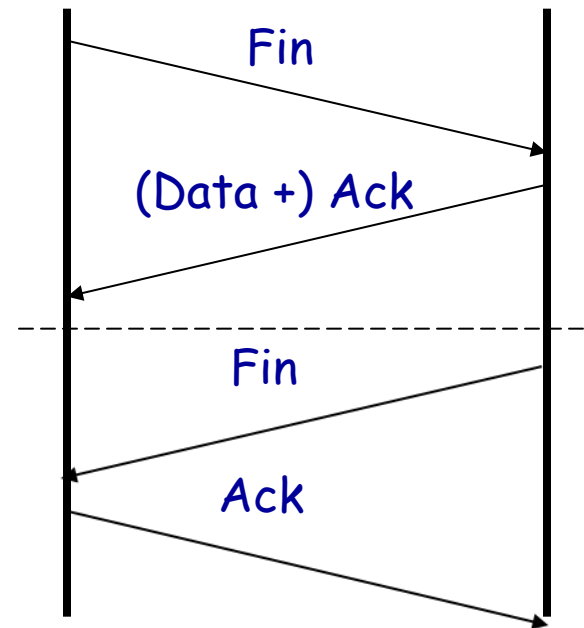


3-way handshake

Liberación

(Activo)
Cliente

(Pasivo)
Servidor

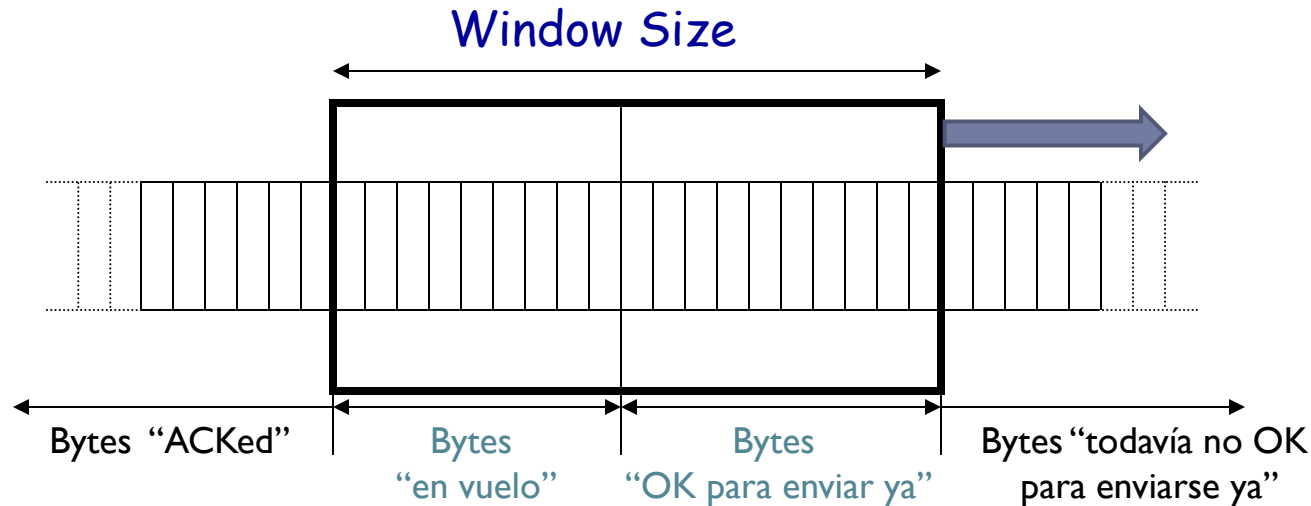


4-way handshake

Caso ideal

En el RFC 793 se plantean varios escenarios

TCP: ventana deslizante

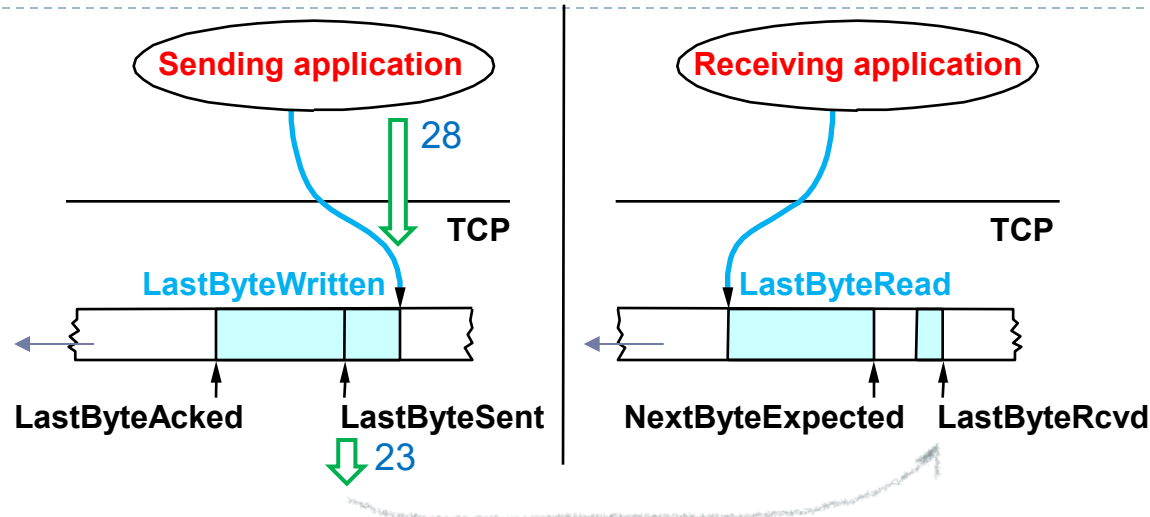


- ▶ Política de retransmisión “Go Back N”
- ▶ Window Size es “advertised” por el receptor (server)
 - ▶ (usualmente de 4KB a 8KB durante la fase de *connection set-up*)

TCP: ventana deslizante

- ▶ Se implementa una variante del **protocolo de ventana deslizante** usado en protocolos de Nivel de Enlace
 - ▶ Garantiza **confiabilidad**
 - ▶ La aplicación recibe los datos en orden
 - ▶ Fuerza el **control de flujo** entre receptor y transmisor

TCP: Ventana Deslizante **Emisor**



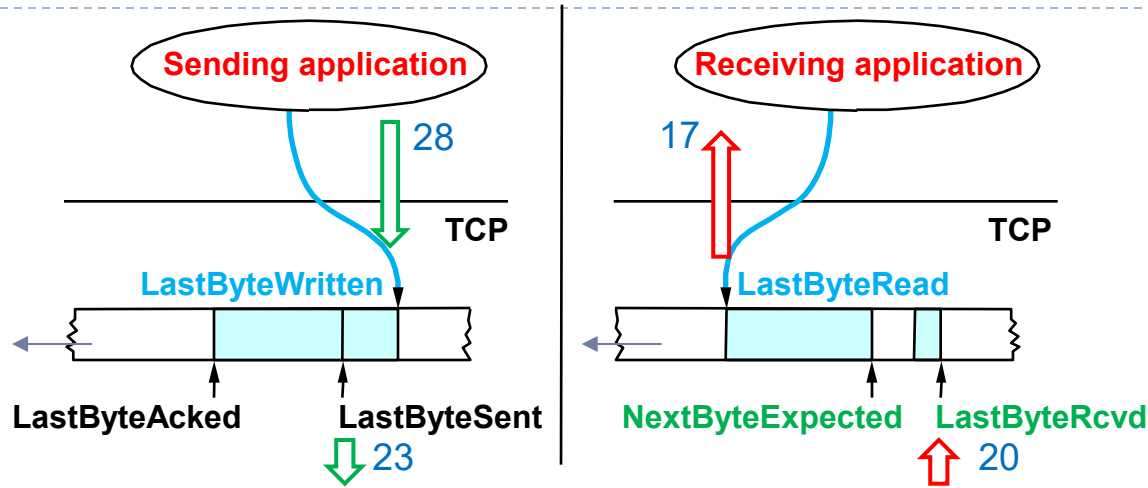
- ▶ Lado **Emisor**: tres punteros, las relaciones son obvias:

$\text{LastByteAcked} \leq \text{LastByteSent}$

$\text{LastByteSent} \leq \text{LastByteWritten}$

- ▶ Se bufferean los bytes entre **LastByteAcked** (los que están a su izquierda ya fueron confirmados) y **LastByteWritten** (los que están a su derecha no fueron aun generados)

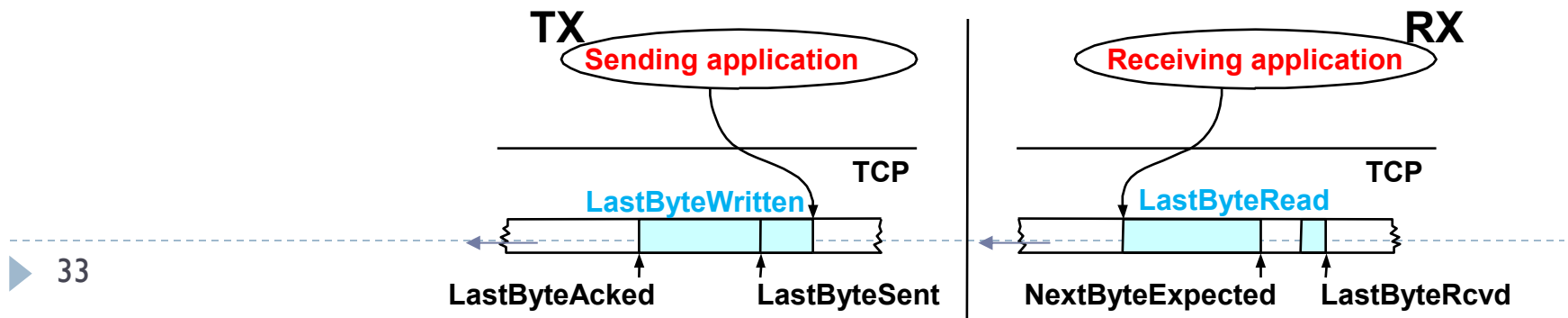
TCP: Ventana Deslizante **Receptor**



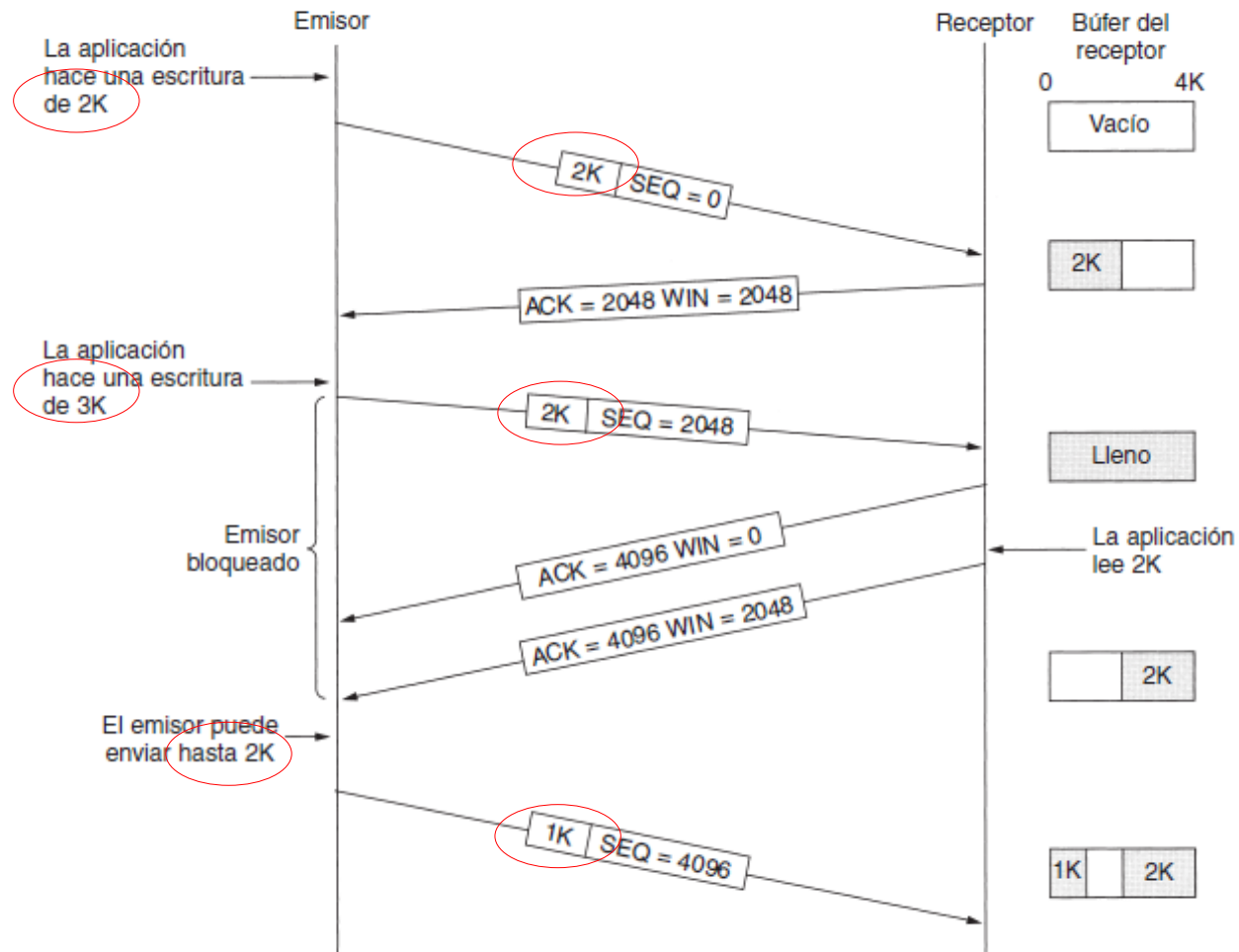
- ▶ Las relaciones son menos intuitivas debido al reordenamiento
 $\text{LastByteRead} < \text{NextByteExpected}$
- ▶ Un byte no puede ser **leído por la aplicación** a menos que éste se haya recibido *y también que todos sus precedentes se hayan recibido*
 $\text{NextByteExpected} \leq \text{LastByteRcvd} + 1$
- ▶ Si están **en orden** se cumple la igualdad. En cambio, si están **en desorden** NextByteExpected apunta al gap de la figura.
- ▶ Se encolan los bytes entre NextByteExpected y LastByteRcvd

TCP: Control de Flujo

- ▶ Tamaño del buffer de **Envío**: **MaxSendBuffer**
- ▶ Tamaño del buffer de **Recepción**: **MaxRcvBuffer**
- ▶ **Lado RX**
 - ▶ $(\text{LastByteRcvd} - \text{LastByteRead}) \leq \text{MaxRcvBuffer}$
 - ▶ $\text{AdvertisedWindow} = \text{MaxRcvBuffer} - (\text{LastByteRcvd} - \text{LastByteRead})$
- ▶ **Lado TX**
 - ▶ $(\text{LastByteSent} - \text{LastByteAked}) \leq \text{AdvertisedWindow}$
 - ▶ $\text{EffectiveWindow} = \text{AdvertisedWindow} - (\text{LastByteSent} - \text{LastByteAked})$
 - ▶ $\text{LastByteWritten} - \text{LastByteAked} \leq \text{MaxSendBuffer}$
- ▶ **Bloquear TX** si $(\text{LastByteWritten} - \text{LastByteAked}) + y > \text{MaxSendBuffer}$, con $y = \text{bytes que se desean escribir}$.
- ▶ **Enviar desde RX** segmentos ACK en respuesta a la llegada de segmentos de datos
- ▶ **Persistir en TX** enviando 1 byte cuando $\text{AdvertisedWindow} = 0$



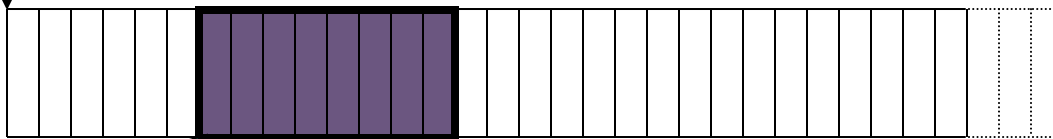
Advertised Window Management



Números de Secuencia

Host A

ISN (initial sequence number)



Seq_Num
= Primer Byte



ACK Seq_Num
= Próximo Byte
esperado



Host B

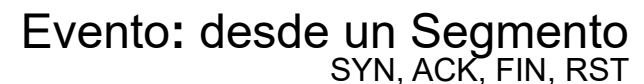




TCP



Máquina de Estados Finitos



Estados del Modelo TCP

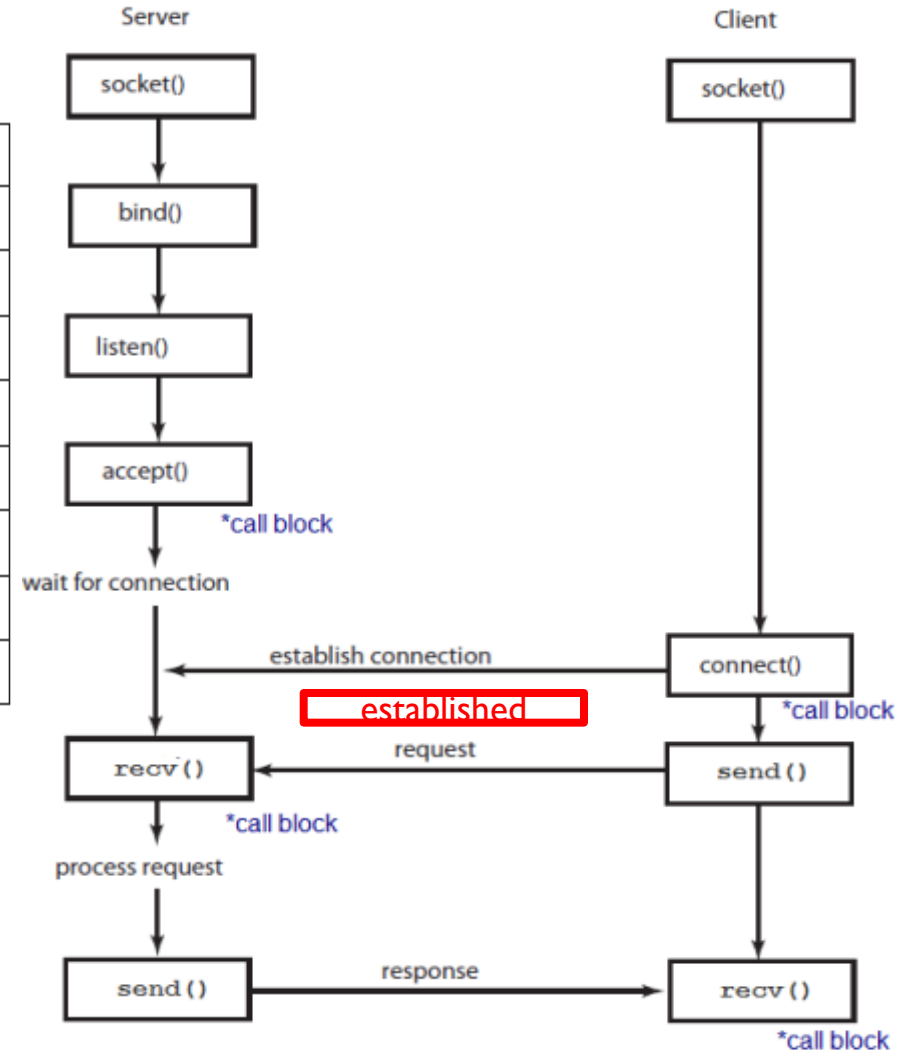
Estado	Descripción
CLOSED	No hay conexión activa ni pendiente
LISTEN	El servidor espera una llamada
SYN RCVD	Llegó solicitud de conexión; espera ACK
SYN SENT	La aplicación comenzó a abrir una conexión
ESTABLISHED	Estado normal de transferencia de datos
FIN WAIT 1	La aplicación dijo que ya terminó
FIN WAIT 2	El otro lado acordó liberar
TIMED WAIT	Espera que todos los paquetes mueran
CLOSING	Ambos lados intentaron cerrar simultáneamente
CLOSE WAIT	El otro lado inició una liberación
LAST ACK	Espera que todos los paquetes mueran

Administración de conexiones

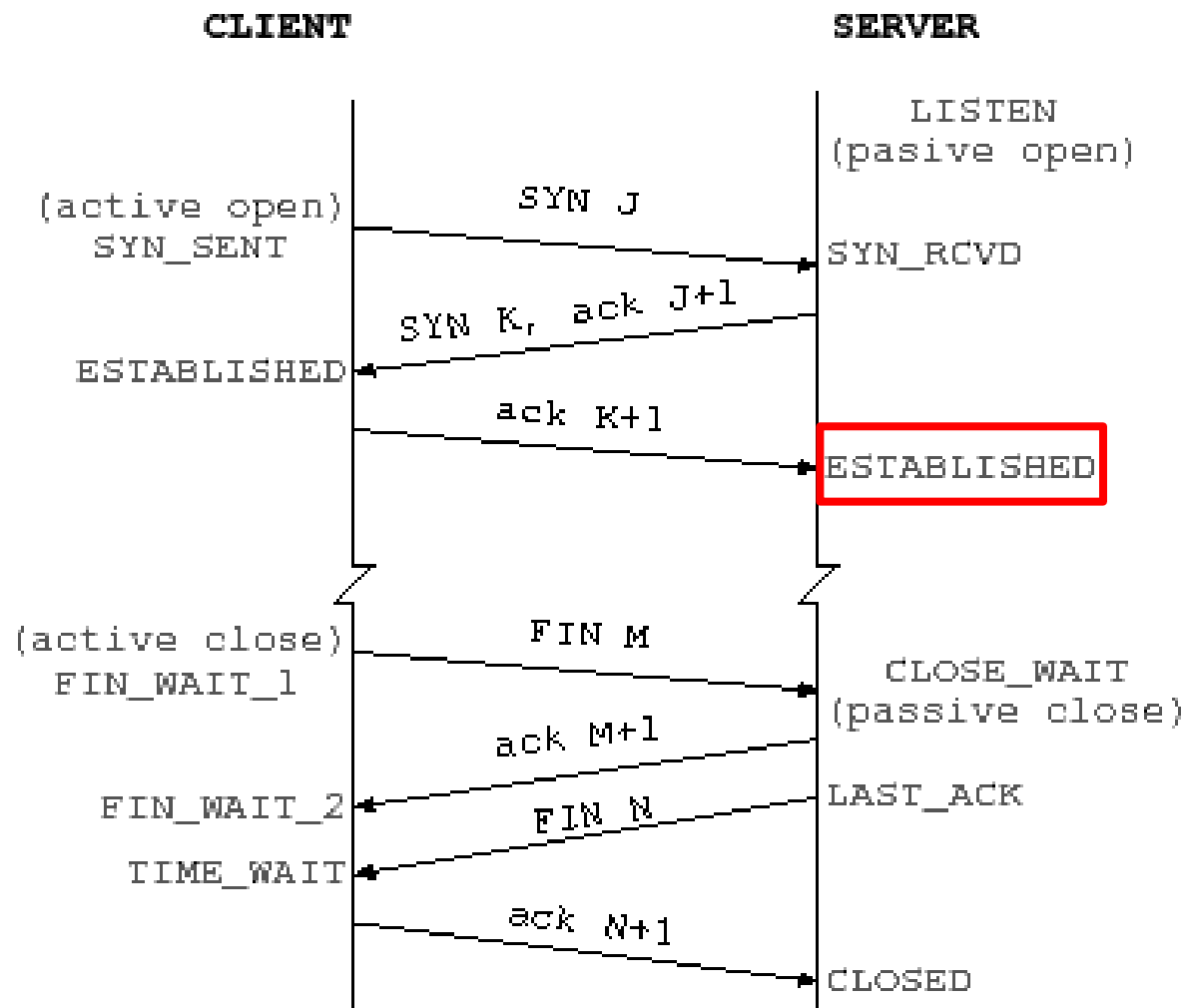
- El caso común de un cliente que se conecta activamente a un servidor pasivo se indica con líneas gruesas (continuas para el cliente, punteadas para el servidor).
- Las líneas delgadas son secuencia de eventos poco comunes.
- Cada línea de la figura se marca mediante un par **evento/acción**.
- El **evento** puede ser una **llamada de sistema** iniciada por el usuario (CONNECT, LISTEN, SEND o CLOSE), o la **llegada de un segmento** (SYN, FIN, ACK o RST) o, en un caso, una expiración de temporizador del doble del tiempo de vida máximo del paquete.
- La *acción* es el envío de un segmento de control (SYN, FIN o RST), o nada, indicado por “-”.
- Los comentarios aparecen entre paréntesis.

Primitivas – sockets berkeley TCP

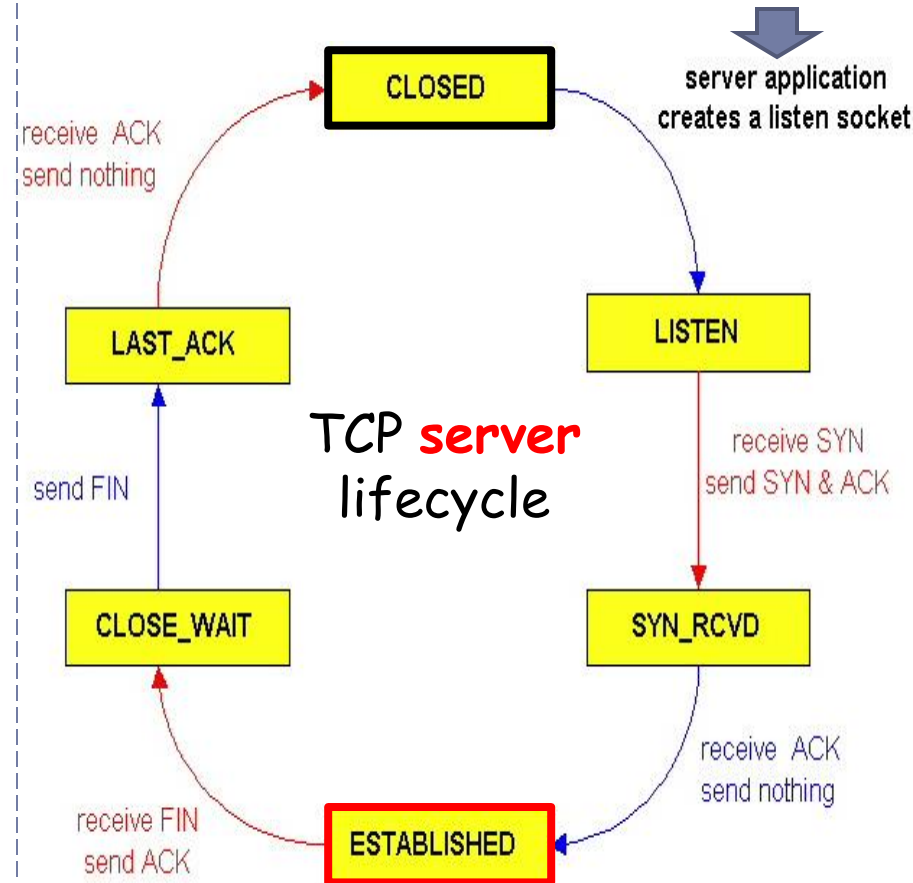
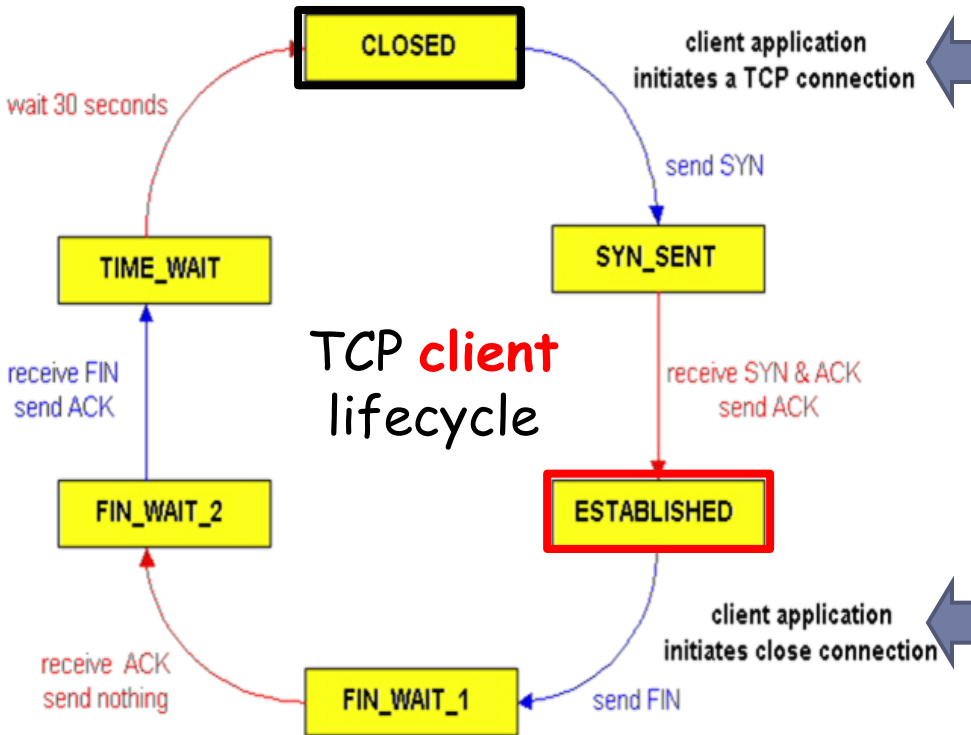
Primitiva	Significado
SOCKET	Crea un nuevo punto terminal de comunicación
BIND	Adjunta una dirección local a un <i>socket</i>
LISTEN	Anuncia la disposición a aceptar conexiones; indica el tamaño de cola
ACCEPT	Bloquea al invocador hasta la llegada de un intento de conexión
CONNECT	Intenta establecer activamente una conexión
SEND	Envía datos a través de la conexión
RECEIVE	Recibe datos de la conexión
CLOSE	Libera la conexión



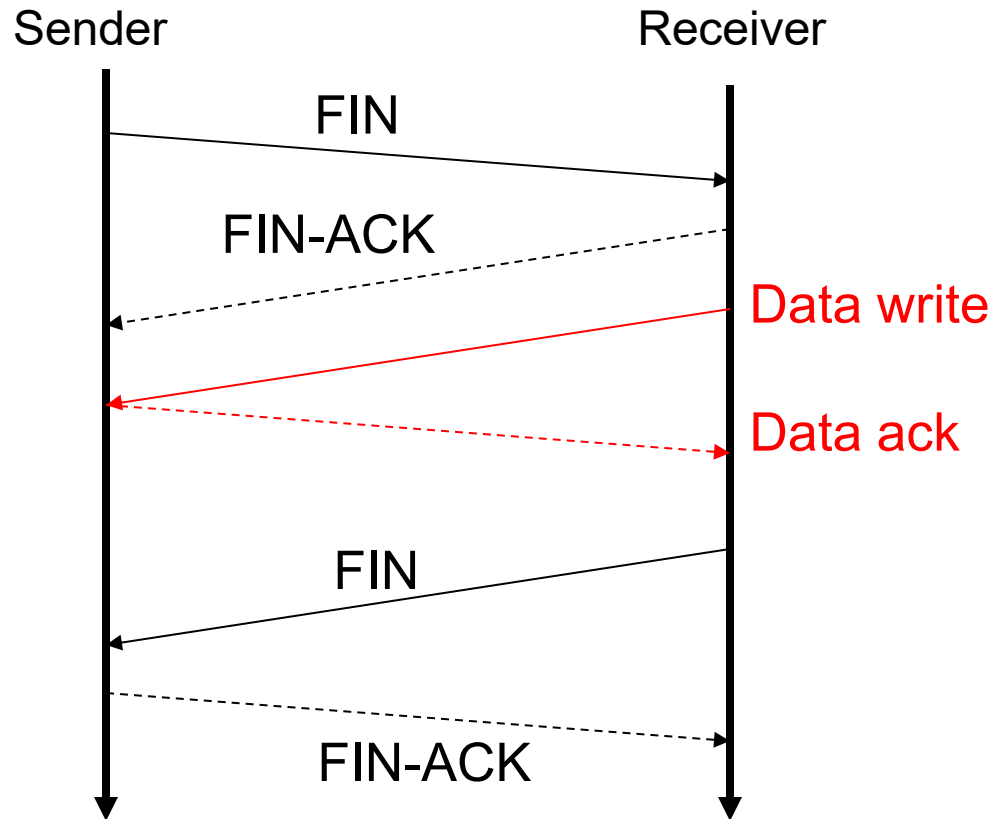
Establecimiento y liberación de una conexión



Ciclos en la Máquina de Estados TCP



Liberación “Tear-down”



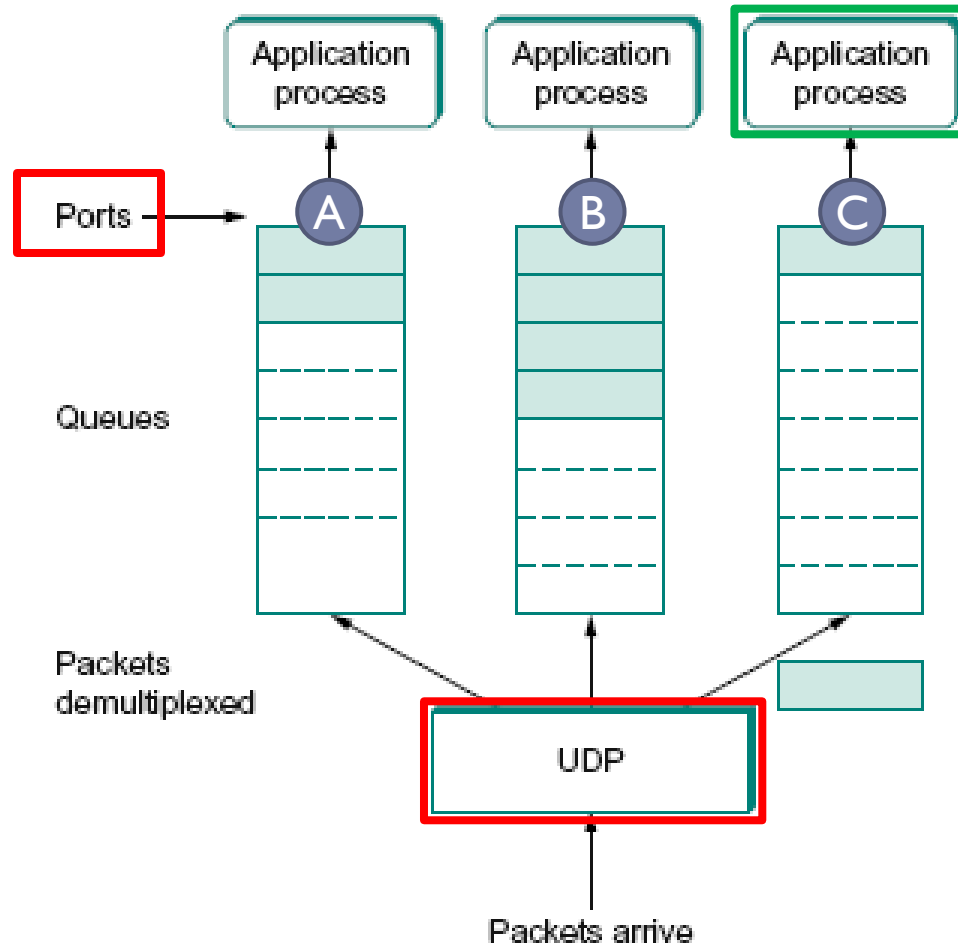


UDP

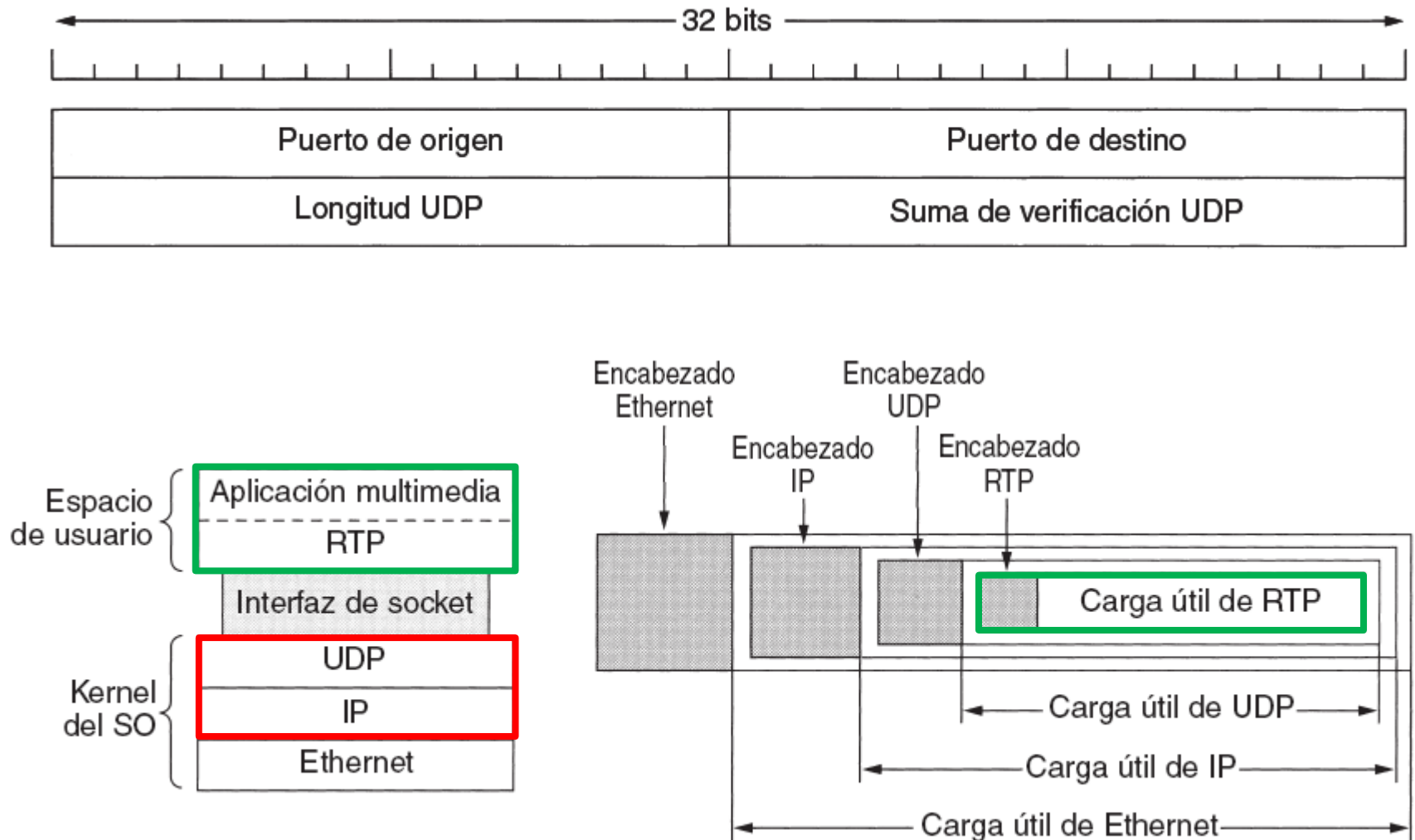


Servicio sin conexión

Multiplexación mediante Puertos



Ejemplo: Transporte UDP y Aplicación RTP





TCP



Retransmisión Adaptativa

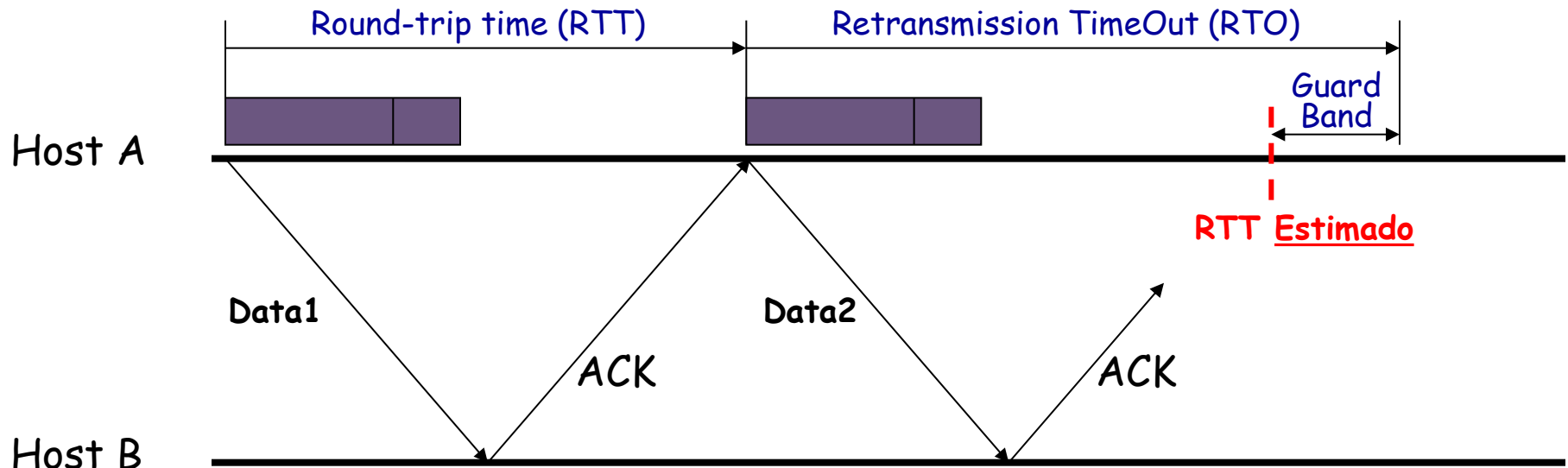
Jacobson (1988)

A A fast algorithm for rtt mean and variation

A.1 Theory

The RFC793 algorithm for estimating the mean round trip time is one of the simplest examples of a class of estimators called *recursive prediction error* or *stochastic gradient* algorithms. In the past 20 years these algorithms have revolutionized estimation and control theory¹⁴ and it's probably worth looking at the RFC793 estimator in some detail.

Retransmisión y Timeouts

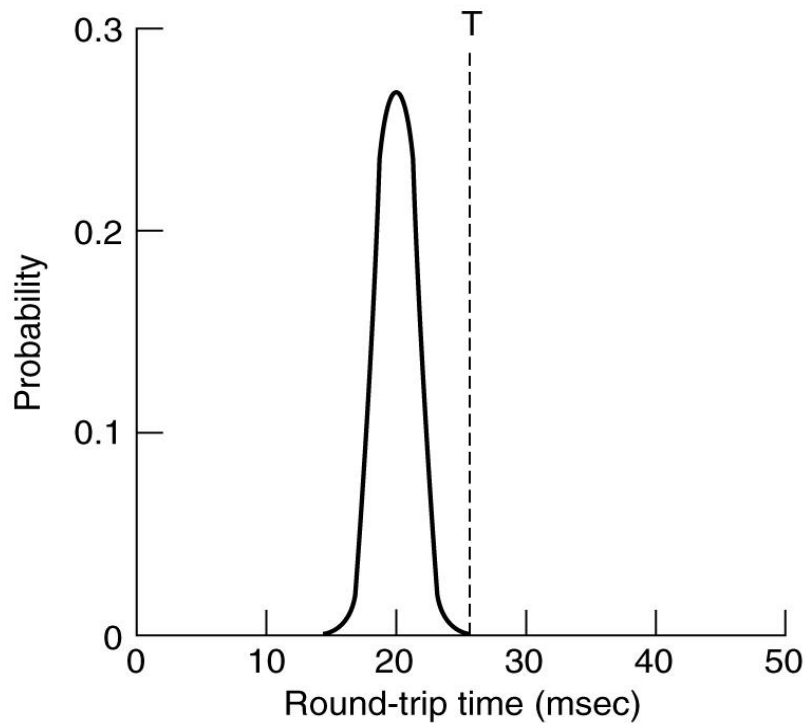


Valor adaptativo de timeout de ReTX (lazo cerrado → teoría de control):

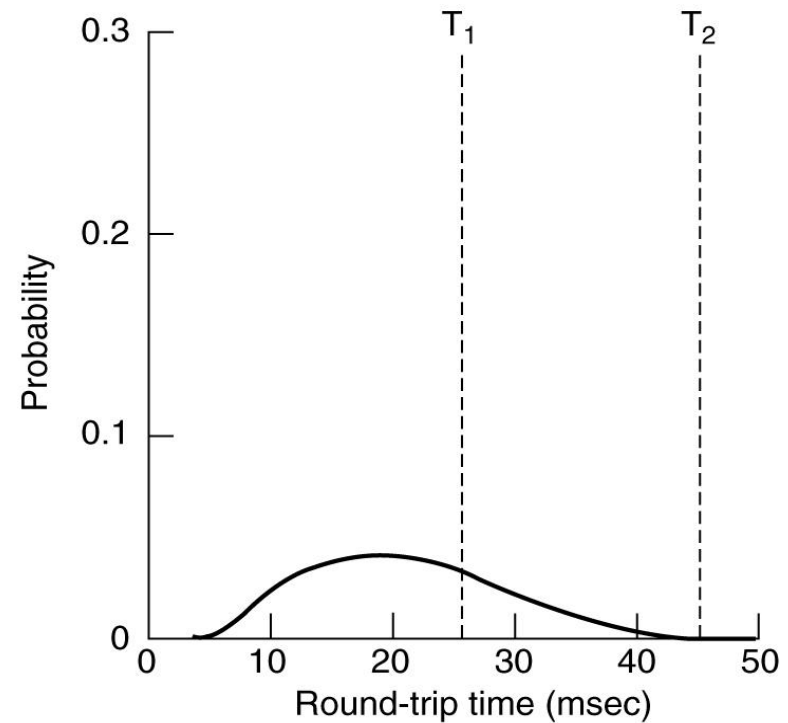
Congestión
Cambios de ruta

} RTT es muy
variable

Concepto: TCP Timer Management



(a)



(b)

(a) Función densidad de probabilidad para tiempos de llegadas de ACK en **Nivel 2**

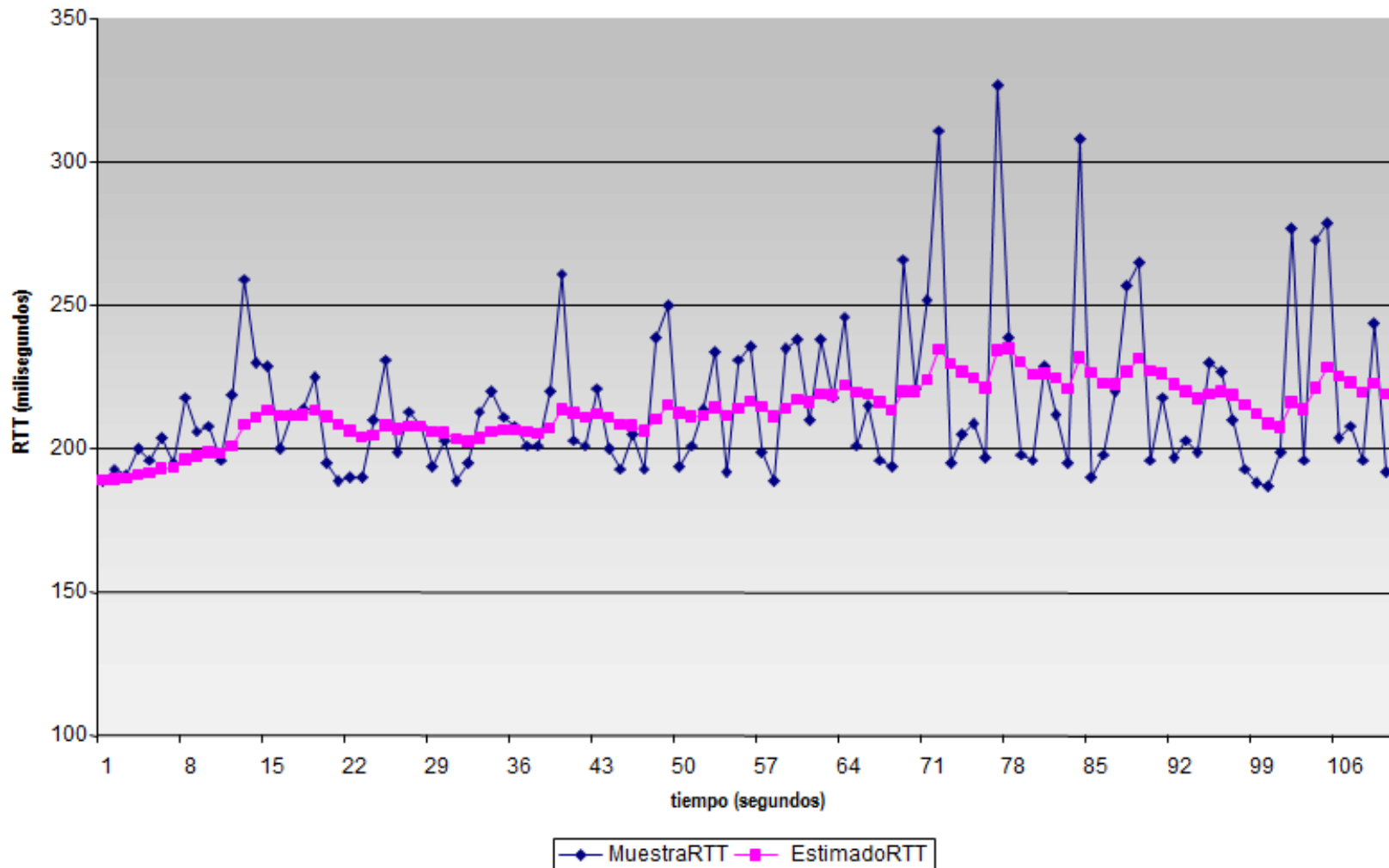
(b) Función densidad de probabilidad para tiempos de llegadas de ACK en **TCP**

Figuras : Tanenbaum 4ta edición

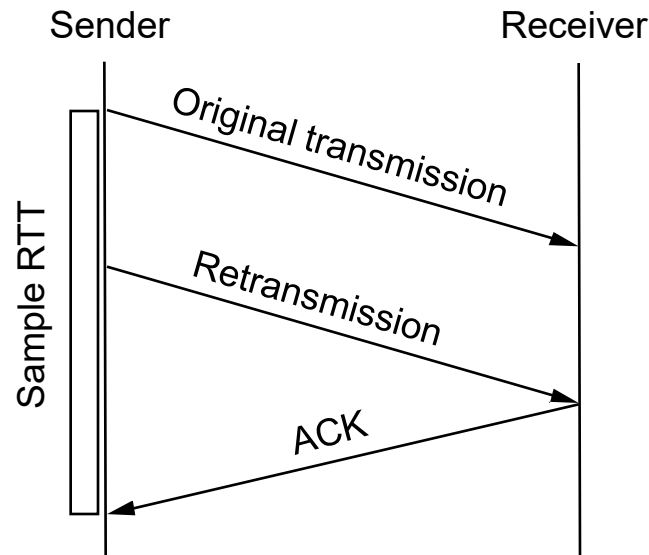
Retransmisión Adaptativa (Algoritmo Original)

- ▶ Mide `SampleRTTi` para cada par segmento / ACK
- ▶ Calcula el promedio ponderado de RTT
 - ▶ $\text{EstimatedRTT} = \alpha * \text{EstimatedRTT} + \beta * \text{SampleRTTi}$
 - ▶ donde $\alpha + \beta = 1$
 - ▶ $0.8 \leq \alpha \leq 0.9$
 - ▶ $0.1 \leq \beta \leq 0.2$
- ▶ Fija `TimeOut` basado en `EstimatedRTT`
 - ▶ $\text{TimeOut} = 2 \times \text{EstimatedRTT}$

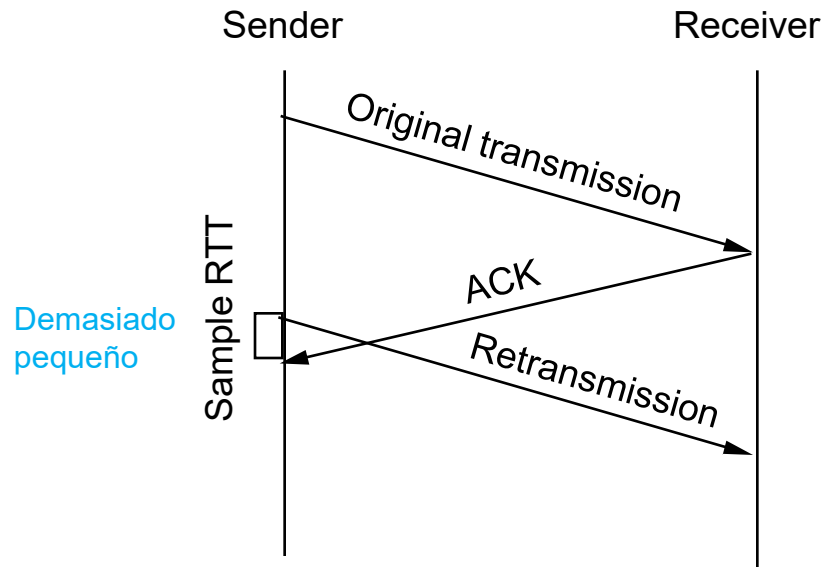
Ejemplo de estimación de RTT



Algoritmo de Karn/Partridge [KP87]



Demasiado largo



Demasiado pequeño

- ▶ Problemas obvios: incertidumbre en los ACKs
- ▶ K/P: No considerar RTT **cuando se retransmite**
 - ▶ Además: Duplicar Timeout luego de cada retransmisión (exponential backoff, un viejo conocido)

Algoritmo de Jacobson/Karels [JK88]

Nueva forma de calcular TimeOut de ReTX considerando la varianza

- ▶ $\text{Diff} = \text{SampleRTT} - \text{EstRTT}$
- ▶ $\text{EstRTT} = \text{EstRTT} + (\delta * \text{Diff})$
- ▶ $\text{Dev} = \text{Dev} + \delta * (|\text{Diff}| - \text{Dev})$
 - ▶ donde δ es un factor entre 0 y 1 (por ejemplo 1/8)
- ▶ Considerar varianza, fijar el timeout
- ▶ $\text{TimeOut} = \mu * \text{EstRTT} + \phi * \text{Dev}$
 - ▶ donde $\mu = 1$ y $\phi = 4$
- ▶ **TimeOut** se acerca a **EstRTT** o a **Dev** dependiendo del valor dinámico de la varianza
- ▶ **Observaciones:**
 - ▶ Los algoritmos son tan buenos/malos como la granularidad de su reloj (500ms en Unix-like)
 - ▶ Un mecanismo preciso de timeout es importante para controlar la **congestión** (como veremos en una próxima clase)
 - ▶ Además de ayudar a controlar la congestión, la idea es no retransmitir cuando no es absolutamente necesario.

RFC 6298 "Computing TCP's Retransmission Timer"

- ▶ El algoritmo básico del emisor TCP mantiene dos variables de estado para computar el RTO
 - **SRTT** (smoothed round-trip time) and
 - **RTTVAR** (round-trip time variation)

Hasta que el emisor realice la medición **para el primer segmento**:

- ▶ “SHOULD set” **RTO** <- 1 segundo (versiones anteriores 3 segundos o más)

Cuando se realiza **la primera medida R**; RTT el host “debe” setear

- **SRTT** <- R
- **RTTVAR** <- R/2
- **RTO** <- **SRTT + max (G, K*RTTVAR)** , siendo K=4, G=1 seg (granularidad)

RFC 6298 "Computing TCP's Retransmission Timer"

- ▶ En las subsiguientes mediciones R' del RTT el host debe setear:
 - $RTTVAR \leftarrow (1 - \text{beta}) * RTTVAR + \text{beta} * |SRTT - R'|$ (1)
 - $SRTT \leftarrow (1 - \text{alpha}) * SRTT + \text{alpha} * R'$ (2)

Con $\text{alpha} = 1/8$ y $\text{beta} = 1/4$ (sugeridos en [JK88]).

Nota : el valor de SRTT usado para actualizar RTTVAR en (1) es el valor antes de su propia actualización en (2). RTTVAR y SRTT debe calcularse en el orden precedente.

- ▶ Luego se debe setear el RTO:
 - $RTO \leftarrow SRTT + \max(G, K * RTTVAR)$
- ▶ Si el RTO calculado es menor a 1 segundo entonces debería redondear a 1 según las consideraciones para evaluar cual es un valor conservativo y evitar retransmisiones espúreas.

TCP

Algoritmo de Nagle
Silly Window

Cuando las aplicaciones leen y/o escriben de forma “ineficiente”

► Envío de pocos bytes

- Considerar una conexión telnet a un editor en el servidor que reacciona con cada letra tipeada (echo), al llegar el carácter a la entidad TCP emisor, se crea un segmento de 21 bytes, a IP 41 bytes, del lado receptor un segmento TCP, mas IP 40 bytes de ACK
 - El editor leyó el carácter envía una actualización de ventana de 40 bytes
 - Una vez que lo proceso, echo del carácter, otros 41 bytes
 - 162 Bytes y cuatro segmentos por cada letra tipeada
-
- Algunas soluciones del lado del receptor: retardar el ACK y de las actualizaciones de ventana
 - Sin embargo, cuando envío paquetes con un byte sigue siendo ineficiente

Algoritmo de Nagle (1984)

- ▶ Cuando la aplicación tiene datos para enviar
 - ▶ If el tamaño de la ventana y los datos disponibles en el buffer $\geq$ MSS
 - Enviar un segmento full
 - ▶ Else
 - If los datos en vuelo están sin reconocer
 - bufferear el nuevo dato hasta que llegue el ACK
 - Else
 - enviar todo los datos nuevos ahora

TCP: persist timer

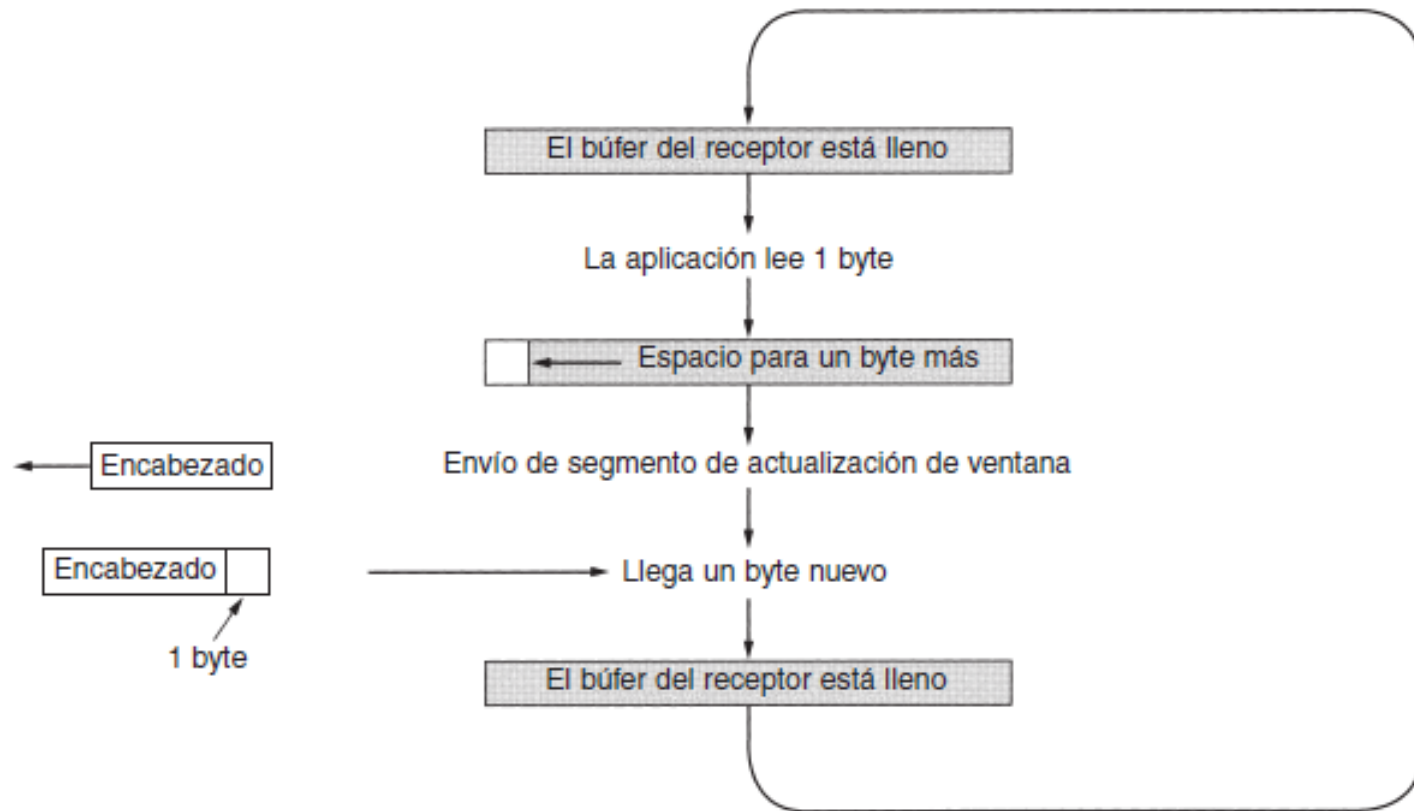
- ▶ Cuando la aplicación lee despacio
 - ▶ Si la app receptora lee muy despacio el buffer se puede llenar y anuncia $W=0$ tamaño de la ventana receptora
 - ▶ Cuando la ventana vuelve a aumentar el receptor envía un segmento ACK con la nueva ventana
 - ▶ Que pasa si se pierde?
 - El emisor no puede enviar datos
 - El receptor no sabe que el ACK se perdió

Solución: TCP envía periódicamente **window probes** con 1 byte de datos

Síndrome de Ventana estúpida (Silly)

- ▶ Se envían datos en bloques y el RX con su app lee muy despacio, por ejemplo un 1 byte a la vez
- ▶ Tomemos el siguiente ejemplo donde inicialmente el buffer del RX está lleno y el emisor lo sabe ya que recibió $W=0$

Silly Window Syndrome (*)



“El buffer ahora está lleno, por lo que el receptor confirma la recepción del segmento de 1 byte pero establece la ventana en 0. Este comportamiento puede continuar indefinidamente”

Solución de Clark

- ▶ No enviar aviso de ventana para 1 Byte
- ▶ Esperar hasta tener una cantidad de buffer considerable
- ▶ Buffer = MSS que se anuncio en la conexión
- ▶ O a la mitad de la capacidad del buffer, o el valor mas pequeño de los dos.

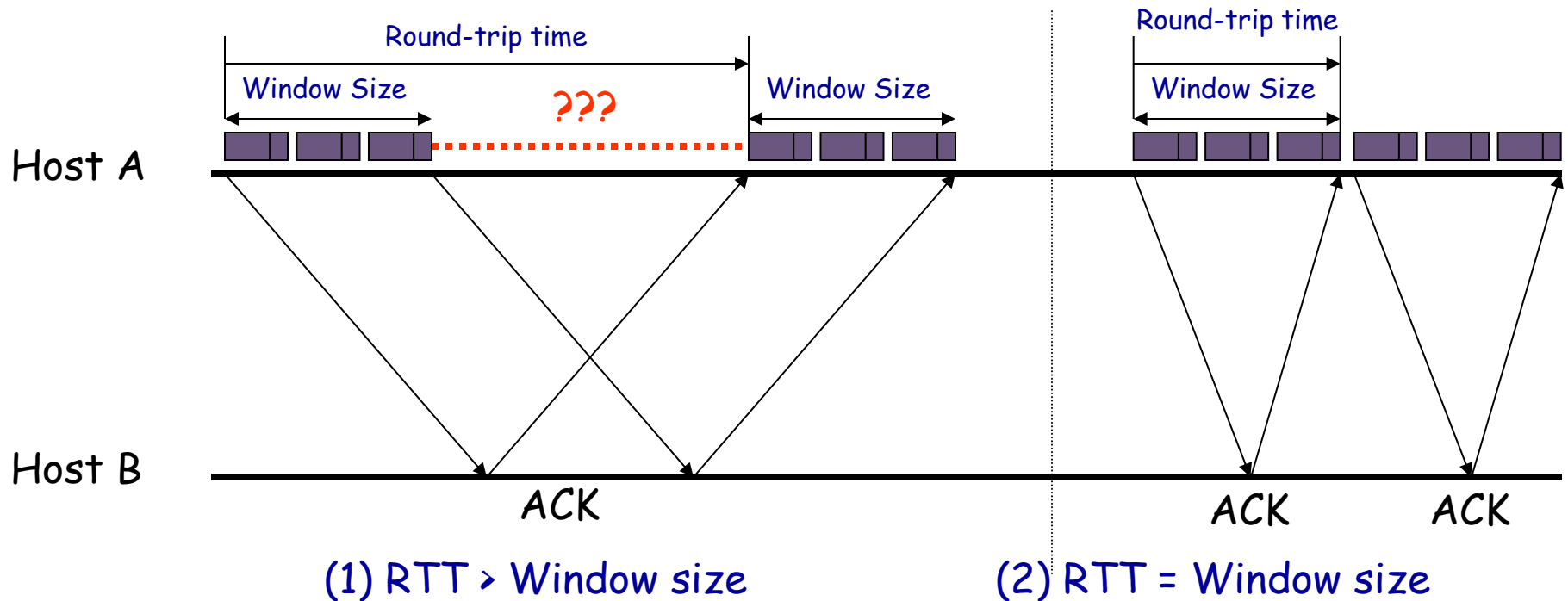


TCP



Uso del ancho de banda
Sockets

TCP Sliding Window



Mantener el “caño” lleno

► AdvertisedWindow de 16 bits (64 KB)

Bandwidth	Delay x Bandwidth Product
T1 (1.5 Mbps)	18KB
Ethernet (10 Mbps)	122KB
T3 (45 Mbps)	549KB
FDDI (100 Mbps) 1.2MB	
STS-3 (155 Mbps)	1.8MB
STS-12 (622 Mbps)	7.4MB
STS-24 (1.2 Gbps)	14.8MB

Asumiendo RTT de 100 ms